



UNIVERSITAS
GADJAH MADA

Configuration Drift pada Deployment Aplikasi Multi-Service di Kluster Kubernetes

Evaluasi Empiris Paradigma Manual dan GitOps
(Studi Kasus InvenioRDM)

Muhammad Argya Vityasy

NIM: 23/522547/PA/22475

Dosen Pembimbing: Dr. techn. Guntur Budi Herwanto, S.Kom., M.Cs.

Dosen Penguji 1: Dr. techn. Kabul Kurniawan, S.Kom., M.Cs.

Dosen Penguji 2: Aina Musdholifah, S.Kom., M.Kom., Ph.D.

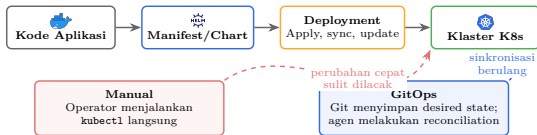
ugm.ac.id

LOCALLY ROOTED, GLOBALLY RESPECTED

Apa Itu Deployment Aplikasi?

Deployment adalah proses membawa aplikasi dari kode menjadi layanan yang berjalan: membangun, mengemas, mengirim, dan menjalankan di kluster. **Aplikasi multi-service** memiliki banyak komponen yang saling bergantung. InvenioRDM, misalnya, terdiri dari 16+ komponen: web, worker, database, search, cache, object storage, ingress, secrets, dan monitoring. **Dua paradigma deployment:**

- **Manual:** operator menjalankan `kubectl` langsung ke kluster.
- **GitOps:** Git menyimpan desired state; agen software melakukan reconciliation secara berkala.



Konteks: Kubernetes dan Masalah Drift

Situasi yang sering terjadi

- Operator menjalankan `kubectl scale` langsung di kluster.
- Dokumentasi tidak diperbarui.
- Tidak ada yang tahu mengapa replika berubah.

Itulah **configuration drift**: *desired state* $\neq$ *actual state*. Dampak: inkonsistensi, pemulihan lambat, audit tidak jelas.



Drift = kluster menyimpang dari apa yang tertulis. Tim kehilangan kepastian tentang state sistem.

Bukti: Drift Bukan Masalah Teoritis

Bukti empiris

- 71% dari 2.260 skrip K8s mengandung *misconfiguration* bermakna (Zhang, 2026).
- 11 kategori *security misconfiguration* berulang (Rahman, 2023).

Tiga penyebab utama drift (Pohjola, 2025)

1. Perubahan langsung tanpa source of truth.
2. Prosedur deployment tidak konsisten.
3. Tidak ada mekanisme deteksi otomatis.

Drift bukan masalah teoritis; ini ancaman operasional nyata terhadap keandalan dan keamanan layanan.

Konsekuensi nyata di klaster akademis

Tim IT universitas yang mengelola repositori data penelitian harus memulihkan layanan setelah drift. Tanpa deteksi otomatis, drift menumpuk sampai insiden.

Saat insiden: apa yang berubah, kapan, mengapa? Tanpa audit trail, pemulihan memakan waktu dan akar masalah luput.

GitOps Menjanjikan Jawaban

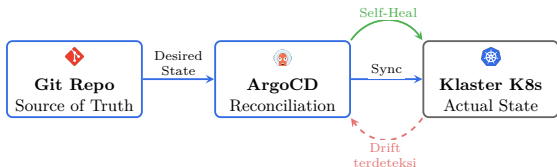
Empat prinsip OpenGitOps

1. Sistem dideklarasikan secara deklaratif.
2. Desired state tersimpan dan berversi di Git.
3. Perubahan ditarik secara otomatis oleh agen software.
4. Agen software secara kontinu mendeteksi dan mengoreksi deviasi.

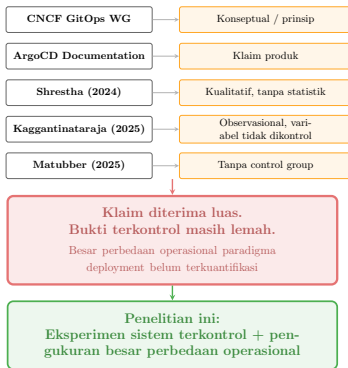
Konsekuensi: drift menjadi *observable* dan dapat dipulihkan otomatis (*self-heal*); audit trail terjamin lewat Git history.

Pertanyaan ilmiahnya bukan “apakah GitOps populer?”, melainkan seberapa besar perbedaan operasionalnya saat drift terjadi.

Prinsip GitOps

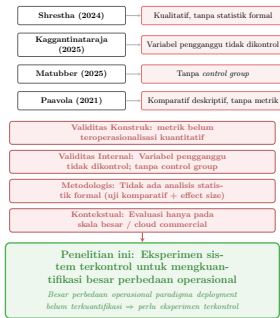


Klaim Industri vs. Bukti Ilmiah



Klaim diterima luas. Bukti terkontrol masih lemah. Besar perbedaan operasional paradigma deployment belum terkuantifikasi secara empiris.

Kesenjangan Penelitian



Empat kesenjangan: metrik belum teroperasionalisasi, variabel belum dikontrol, uji statistik belum dilakukan, evaluasi belum menyentuh konteks klaster akademis.

Tanpa bukti terkontrol, tim IT memilih paradigma deployment berdasarkan keyakinan, bukan data tentang besar perbedaan operasional.

Sejauh mana paradigma deployment manual dan GitOps menghasilkan perbedaan pada karakteristik operasional ketika *configuration drift* terjadi?

Tiga dimensi pengukuran

1. **R1:** Konsistensi: apakah state kluster kembali ke yang diharapkan setelah drift?
2. **R2:** Waktu: berapa lama drift terdeteksi dan dipulihkan?
3. **R3:** Identifikasi: berapa lama sumber drift dapat diidentifikasi?

Skenario pengujian Tujuh skenario drift terkontrol (A–G) yang merepresentasikan kesalahan operasional umum pada Kubernetes: perubahan replika, image, ConfigMap, resource deletion, patch manual, drift lintas namespace, dan perubahan Secret. **Pendekatan analisis** Eksperimen sistem terkontrol: mengukur besar perbedaan operasional melalui effect size dan confidence interval, dengan uji statistik sebagai analisis pendukung.

Kita menguji satu pertanyaan dalam satu konteks yang terdefinisi:

1. Satu kluster Kubernetes Rancher: 3 node, 24 inti CPU, ~23 Gi memori.
2. InvenioRDM (16+ komponen) sebagai subjek uji, bukan objek penelitian.
3. Tujuh skenario *drift* terkontrol (A--G).
4. Dua paradigma: deployment manual (`kubectl`) dan GitOps (ArgoCD).
5. Analisis: Shapiro-Wilk, *t-test* / *Mann--Whitney U*, $\alpha = 0,05$, effect size (Cohen's d / Cliff's δ), confidence interval 95%.
6. Di luar lingkup: *multi-cluster* dan Git tanpa agen *reconciliation*.

Skenario F bukan skenario drift konvensional; ia menguji batas visibilitas GitOps pada resource di luar Application boundary.

Tujuan dan Manfaat Penelitian

Tujuan

1. Mengukur konsistensi deployment setelah drift (R1).
2. Mengukur *detection time* dan *recovery time* (R2).
3. Mengukur *Mean Time To Identify* --- waktu identifikasi sumber drift (R3).

Teoritis

Bukti empiris terkontrol tentang besar perbedaan operasional antara dua paradigma, bukan klaim vendor.

Metodologis

Protokol yang dapat direplikasi: skenario, metrik, dan analisis statistik.

Praktis

Data untuk tim IT memilih paradigma deployment dan mengetahui skenario drift mana yang paling rentan.

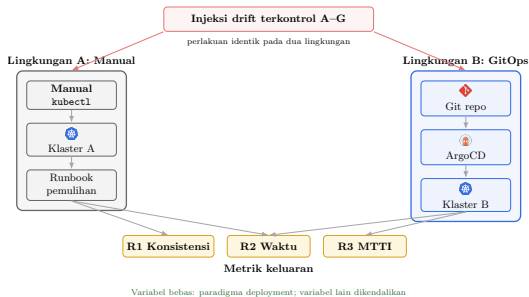
Kontribusi utama: mengkuantifikasi besar perbedaan operasional pilihan paradigma, bukan membuktikan satu teknologi selalu menang.

Penelitian Terkait dan Pembeda

Studi sebelumnya mendukung potensi GitOps, tapi belum ada yang mengisolasi efek paradigma dari variabel:

Sumber	Temuan Utama	Kelemahan Metodologis
Shrestha (2024)	GitOps membantu deteksi drift	Penilaian kualitatif; tanpa statistik
Kaggantinaraja (2025)	Deklaratif lebih cepat dan aman	Variabel pengganggu tidak dikontrol
Matubber (2025)	Reconciliation time dapat diukur	Tanpa <i>control group</i>
Paavola (2021)	ArgoCD cocok untuk banyak aplikasi	Komparatif deskriptif; tanpa metrik kuantitatif
Damore (2021)	ArgoCD pada hybrid cloud	Studi implementasi; tanpa evaluasi metrik

Pembeda: eksperimen terkontrol, variabel dikendalikan, uji statistik formal, metrik didefinisikan sebelum eksperimen.



Yang disamakan

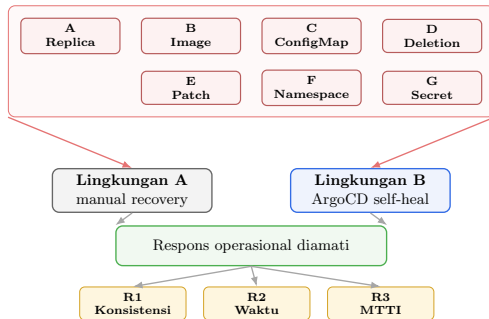
Workload, skenario drift, klaster, dan prosedur pengukuran identik. Satu-satunya yang dibedakan: paradigma deployment.

Mengapa InvenioRDM

16+ komponen yang saling bergantung, infrastruktur luas (database, search, cache, ingress, secrets), dan *open-source* dari CERN. Subjek uji, bukan objek penelitian.

Tujuh Skenario Drift yang Sama Disuntikkan ke Dua Lingkungan

Tujuh skenario drift yang sama diberikan ke dua paradigma



Kita menyuntikkan drift yang sama, lalu membandingkan bagaimana masing-masing paradigma merespons. Itulah variabel dependen kita.

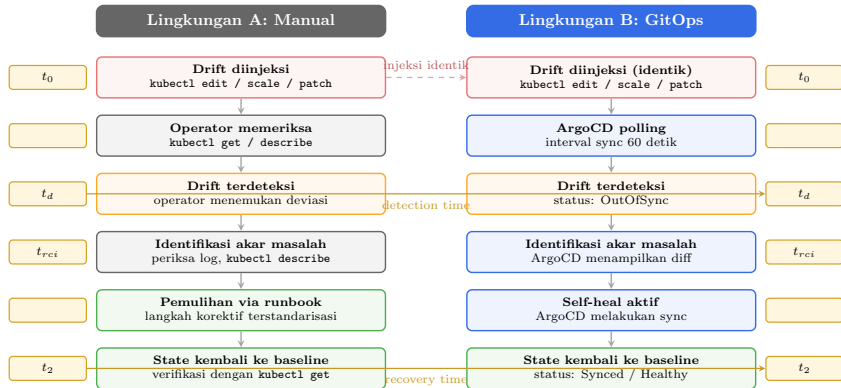
Metrik Pengukuran: Apa yang Kita Ukur dan Bagaimana

Metrik	Operasionalisasi	Output
R1 Konsistensi	% resource cocok dengan desired state	Nilai % pada baseline, saat drift, setelah recovery
R2a Detection time	$t_d - t_0$	Waktu dari injeksi hingga drift terdeteksi
R2b Recovery time	$t_2 - t_0$	Waktu dari injeksi hingga state kembali baseline
R3 MTTI	$t_{id} - t_d$	Waktu dari deteksi hingga identifikasi akar masalah

R1, R2, R3: metrik kuantitatif berbasis *timestamp*. Utama: effect size dan confidence interval. Pendukung: uji statistik (*t*-test / Mann-Whitney U).

R1, R2, R3: metrik kuantitatif berbasis *timestamp*. Utama: effect size dan confidence interval. Pendukung: uji statistik (*t*-test / Mann-Whitney U).

Prosedur Eksperimen: Dari Drift Injection sampai Pemulihan



Setiap skenario diulang $n = 5$ (R1) atau $n = 10$ (R2) kali per lingkungan; urutan skenario diacak untuk menghindari efek pembelajaran.

Hierarki interpretasi

1. **Effect size** (Cohen's d / Cliff's δ) → **utama**
2. **Confidence interval 95%** → **utama**
3. p -value (t -test / Mann--Whitney U) → pendukung

Mengapa effect size di atas?

- ⇒ Keputusan deployment bergantung pada *significance praktis*, bukan semata p -value.
- ⇒ "Seberapa besar perbedaan operasionalnya?" bukan "Apakah perbedaannya signifikan?"

Uji normalitas: Shapiro-Wilk.

Ukuran sampel dan power

- Power = 0,80, $\alpha = 0,05$, target efek besar ($d = 1,0$).
- Minimum $n = 17$ per grup secara total.
- **R2**: $7 \times 10 = 70$ observasi per lingkungan ($4 \times$ lipat minimum).
- **R1, R3**: $7 \times 5 = 35$ observasi per lingkungan ($2 \times$ lipat minimum).
- Jika efek sedang teramati, pengulangan ditambah.

Kontribusi utama: besar efek dan significance praktis, bukan semata p -value

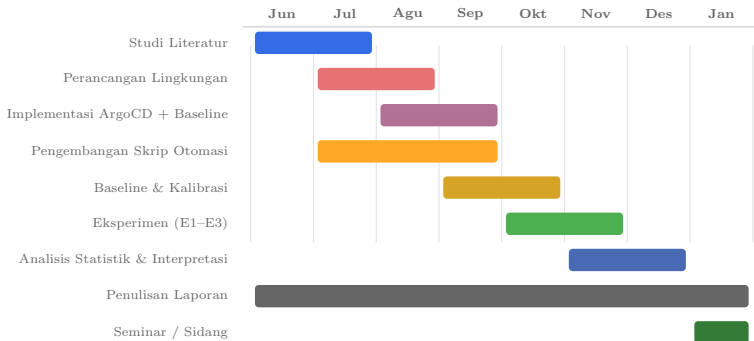
Ancaman terhadap Validitas dan Mitigasi

Kita mengakui ancaman dan mengontrol yang bisa dikontrol:

Ancaman	Mitigasi dalam rancangan
Operator tunggal Efek pembelajaran	Recovery runbook dan timestamp otomatis Urutan <i>eksekusi</i> skenario diacak (A-F-C-B-G-A-D-...), bukan jenis drift, paradigma, atau prosedur recovery
Instrumentasi berubah Satu workload Satu alat GitOps	Versi Kubernetes, ArgoCD, dan workload dikunci InvenioRDM mewakili kelas workload multi-service ArgoCD adalah implementasi GitOps yang paling umum

Klaim dibatasi pada konteks yang didefinisikan. Ini bukan studi tentang semua kluster Kubernetes, tapi tentang satu pertanyaan di satu konteks terkontrol.

Jadwal Penelitian



Juni 2026 – Januari 2027 (8 bulan)

8 bulan: Juni 2026 sampai Januari 2027. Eksperimen inti berlangsung selama Oktober dan November.

Kontribusi yang Diharapkan

1. **Bukti empiris terkontrol:** kita akan tahu *seberapa besar* perbedaan operasional ketika drift terjadi pada dua paradigma deployment --- seberapa cepat, seberapa konsisten, seberapa mudah didiagnosis --- bukan hanya arah perbedaan.
2. **Protokol yang dapat direplikasi:** tujuh skenario drift terstandarisasi, tiga metrik teroperasionalisasi, dan prosedur analisis yang sudah ditetapkan sebelum eksperimen.
3. **Masukan operasional berbasis data:** tim IT universitas yang harus memilih antara melatih operator pada prosedur manual atau menginvestasikan waktu membangun GitOps akan punya data tentang *besar perbedaan* berdasarkan skenario drift mana yang paling berdampak pada masing-masing paradigma.

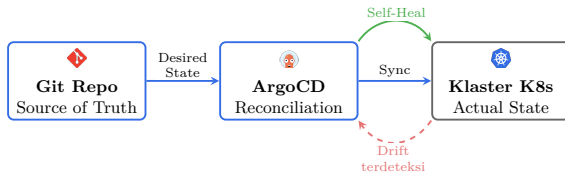
Tanpa kuantifikasi terkontrol, keputusan deployment hanya berdasarkan keyakinan. Dengan effect size yang terukur, berdasarkan data.

Backup: ArgoCD dan Mekanisme Reconciliation

- Git sebagai *single source of truth*
- Konfigurasi dideklarasikan secara deklaratif
- Agen *software* melakukan *reconciliation*
- Drift menjadi *observable* dan dapat dipulihkan otomatis

ArgoCD terdiri dari tiga komponen: *application controller*, *repo server*, dan *Redis cache*.

Prinsip GitOps



Backup: MTTI --- Mean Time To Identify

Pertanyaan penguji: Bagaimana Anda mengukur waktu identifikasi secara objektif?

Definisi operasional MTTI:

- $MTTI = t_{id} - t_d$: waktu dari deteksi drift hingga sumber drift berhasil diidentifikasi.
- Identifikasi dianggap selesai ketika operator menyebutkan **(1)** kategori drift dan **(2)** sumber daya terdampak, lalu jawaban diverifikasi terhadap skenario injeksi yang diketahui.

Mengapa MTTI menggantikan rubrik traceability:

- Menghasilkan *timestamp* objektif, bukan penilaian subjektif.
- Langsung mengukur usaha diagnostik operasional.
- Terkait langsung dengan *incident response*.
- Mudah dijelaskan dan mudah dipertahankan.

Perbedaan antar-paradigma:

- Lingkungan B (GitOps): ArgoCD menampilkan *diff*, akar masalah langsung terlihat.
- Lingkungan A (Manual): operator memakai `kubectl describe` dan log, memerlukan pencarian manual.

• *Timestamp* dicatat secara otomatis oleh skrip saat jawaban benar disebutkan

Backup: Mengapa MTTI Menggantikan Rubrik dan Intervensi Manual?

Pertanyaan penguji: Mengapa tidak mempertahankan rubrik traceability dan hitungan intervensi manual?

Intervensi manual (dihapus):

- Hasilnya sebagian besar sudah ditentukan: manual > 0 , GitOps ≈ 0 .
- Memberikan sedikit informasi baru di luar apa yang sudah diketahui dari desain paradigma.
- Menghapusnya membuat desain penelitian lebih bersih dan fokus.

Rubrik traceability (dihapus):

- Skala 1--5 rentan terhadap pertanyaan: mengapa skala ini? apakah rubrik benar-benar mengukur traceability?
- Cohen's K menambah kompleksitas tanpa menambah kejelasan substantif.

MTTI menggantikan keduanya:

- Mengukur aspek yang paling penting dari traceability: berapa lama sumber drift bisa diidentifikasi

Backup: Mengapa InvenioRDM, Bukan Workload Lain?

Pertanyaan penguji: Mengapa tidak Online Boutique, Sock Shop, atau WordPress?

Tiga kriteria pemilihan:

1. **Dependensi kompleks:** 16+ komponen yang saling bergantung dan memiliki urutan deployment, drift pada satu komponen bisa merambat.
2. **Infrastruktur luas:** database, *search*, *cache*, *object storage*, *ingress*, TLS, *secrets*, monitoring, dan backup.
3. **Reprodusibilitas akademis:** *open-source*, dikembangkan CERN, dan digunakan di institusi akademis.

InvenioRDM adalah workload uji; klaim penelitian dibatasi pada workload multi-service dengan karakteristik serupa.

Backup: Operator Anda Juga Penelitinya, Bukan Ancaman Serius?

Pertanyaan penguji: Operator Anda juga penelitinya. Mengapa ini bukan ancaman serius?

Empat mitigasi:

1. Operator mengikuti *recovery runbook* yang telah ditentukan sebelumnya, tidak ada keputusan subjektif selama *recovery*.
2. Seluruh *timestamp* direkam oleh skrip otomasi, bukan oleh penilaian manusia.
3. Metrik yang tidak bergantung pada operator (R1 konsistensi, R2 deteksi otomatis ArgoCD) memberikan pengukuran objektif.
4. Operator tunggal justru **mengurangi** varians antar-*run*, sehingga memperkuat validitas internal perbandingan antar paradigma.

Keterampilan operator memengaruhi kedua paradigma secara sama: operator, runbook, klaster, dan skenario yang sama. Oleh karena itu, perbandingan tetap bermakna.

Backup: Apa Jika Effect Size Kecil?

Pertanyaan penguji: Jika R1, R2, R3 menunjukkan effect size kecil, apakah penelitian ini gagal?

Tiga interpretasi, semua informatif:

1. **Effect kecil:** Kedua paradigma sebanding dalam konteks terkontrol ini. Ini temuan berharga, berarti otomatisasi GitOps bukan keuntungan mutlak di semua skenario.
2. **Effect sedang/besar tapi confidence interval lebar:** Effect ada tapi varians tinggi. Tambah pengulangan untuk mempersempit interval.
3. **R3 (MTTI) tetap memberikan nilai:** Waktu identifikasi sumber drift terjamin secara struktural oleh desain GitOps, terlepas dari besar effect R1 dan R2.

Kontribusi penelitian adalah kuantifikasi yang dapat diperiksa, apa pun besar effect yang teramati.